OPTIMIZATION OF SPARSE ARRAYS BY AN IMPROVED SIMULATED ANNEALING ALGORITHM

Jon-Fredrik Hopperstad

Schlumberger Geco-Prakla,
P. O. Box 234, N-1372 Asker, Norway.
E-mail: hopperstad@slb.com

Sverre Holm

Dep. of Informatics, Univ. of Oslo,
P. O. Box 1080, N-0316 Oslo, Norway.
E-mail: sverre@ifi.uio.no

ABSTRACT

Simulated annealing has earlier been found successful in sparse array optimization. However, finding the array pattern requires a discrete Fourier transform which makes the algorithm time consuming. A faster evaluation of the array pattern allows more element configurations to be searched. This gives the algorithm a larger degree of stochastic freedom, which improves the peak sidelobe level of the final array. Several new sparse arrays are presented, which are an improvement of those found in the literature.

1. INTRODUCTION

Sparsely sampled arrays have been used in several fields such as radar, sonar, ultrasound imaging and seismics. These arrays provide a large aperture with fewer channels. The most common design criteria for sparse arrays are low mainlobe width and small peak sidelobes. Several optimization algorithms have been proposed ranging from dynamic programming, [1], to linear programming, [2], and simulated annealing, [3], [4], [5]. In [3] simulated annealing was used to minimize the maximum sidelobe level for an array with 50λ aperture and 25 active elements lying on a grid with $\lambda/2$ element distance. This is a problem that has been optimized since the sixties, and a table of the solutions obtained is given in Table 1 based on [3].

In this paper we present a simulated annealing procedure which is an improvement over that of [3], [4], [5] in several respects. First of all it is faster since it is implemented from the observation that the simulated annealing algorithm consists in perturbing just a single element at a time. The faster evaluation of the array pattern means that one can evaluate more configurations and therefore the improved simulated annealing algorithm allows perturbation at an arbitrary location in the array, rather than within the interval given by the neighbors on the right-hand and left-hand sides of the element to be perturbed as in [3]. The improved algorithm was used in [6] who obtained ten solutions better than that of [3], where two of them will be presented here in addition to results on a 50×50 array with 500 active elements.

2. THEORY

For an array with M elements, the number of combinations when a subset of K elements are to be picked are:

$$\binom{M}{K} = \frac{M!}{(M-K)!K!} \tag{1}$$

Assuming that the elements are unweighted, the one-way array pattern is:

$$W(\vec{k}) = \sum_{m=0}^{K-1} e^{j\vec{k}\cdot\vec{x}_m} \tag{2}$$

where $\vec{k}$ is the wavenumber and $\vec{x}_m$ is the position of each element. With a regular spatial sampling distance of $\lambda/2$ in the x and y direction the spatial wavenumber can be expressed as:

$$k_x = \frac{2\pi}{\lambda}\sin\phi = \frac{2\pi}{\lambda}u \tag{3}$$

$$k_y = \frac{2\pi}{\lambda}\sin\theta = \frac{2\pi}{\lambda}v \tag{4}$$

where u and v is the deviation from the angle of look in the x and y direction. In the following results λ is assumed to equal 2π.

3. THE ALGORITHM

Simulated Annealing is a heuristic optimization method which was originally used to find the minimum energy state of the molecules in a substance [9].

Figure 1 shows the sturcture of the simulated annealing algorithm used for array optimization, which is similar to the one presented in [3]. From an initial temperature, T_i, and a initial random element configuration, $\mathbf{X_0}$, the algorithm runs through *num_iter* temperature stages. The

Year	Min sidelobe level	Optimization method	Reference
1964	-8.8 dB	Dynamic programming	[1]
1966	-8.9 dB	Space-tapering	[7]
1968	-10.14 dB	Dynamic programming	[8]
1996	-12.07 dB	Simulated annealing	[3]
1998	-12.36 dB	Simulated annealing	[6]

Table 1: Table of solutions to the problem of finding the best 25 unit weight element positions in a $L = 50\lambda$ aperture.

```
Procedure Simulated_Annealing
begin
    initialize(X_0, T_0)
    for i=1 to num_iter
        T_i = cT_0/i
        for j=1 to num_pert
            X_p=perturbate( X_{j-1})
            ΔE = E(X_p) − E(X_{j-1})
            if ΔE< 0 or exp(−ΔE/T_i) < rnd[0,1]
                X_j = X_p
            else
                X_j = X_{j-1}
            endif
        endfor
    endfor
end
```

Figure 1: The structure of the simulated annealing algorithm. rnd[0,1] returns a random number uniformally distributed on the interval $[0, 1]$. $\mathbf{X_0}$ is the element configuration at the initial temperature, T_0. $E(\mathbf{X_j})$ is the peak sidelobe level of element configuration $\mathbf{X_j}$.

algorithm performs *num_pert* perturbations on the current configuration for each temperature. The perturbated array, $\mathbf{X_p}$, is accepted if this gives a better peak sidelobe level ($\Delta E < 0$) or if the increase in peak sidelobe level is below a statistical threshold.

The main difference between the implementation presented here and the one in [3] is the generation of perturbated arrays. In the latter each element was perturbated sequencially and limited to the available positions between the left-hand and right-hand neighbor. Instead we propose to select perturbation elements randomly and without limitations on the available positions. This gives the algorithm a larger degree of stochastic freedom allowing more array configurations to be reached within one single perturbation.

A second improvement over that of [3], [4], [5] is an increase in speed. The evaluation of (2) requires a discrete Fourier transform that can be implemented by a Fast Fourier Transform in order to speed it up. A further speed increase can be obtained by the observation that the simulated annealing algorithm consists in perturbing just a single element at a time. Therefore the array pattern of the perturbed array can be found by subtracting the contribution of the element that was removed and adding the contribution of that which was added. When all contributions from all the elements at all the angles are precomputed and stored in memory this results in a speed increase. A radix-2 FFT of N real values can be computed with a $N/2$ points FFT [10]. This gives $N/4 \cdot \log_2(N/2)$ complex multiplications and $N/2 \cdot \log_2(N/2)$ complex additions [10]. The proposed method of precomputed values is however less than $2 \cdot N/2$ complex additions (depending on the assumed width of the mainlobe). In the case of 256 points for $u \in [-1, 1]$ the latter method saves over 40% of the additions and all of the 448 complex multiplications.

4. 1D LAYOUT OPTIMIZATION

When a $N = 4096$ point Fourier transform is used for evaluation of the array pattern, the optimal solution of [3] has a sidelobe level of -12.03 dB and a -6 dB beamwidth of $2.10°$ (see Fig. 2). Ten solutions that are slightly better are given in [6]. Two of the better ones are shown in Figs. 3

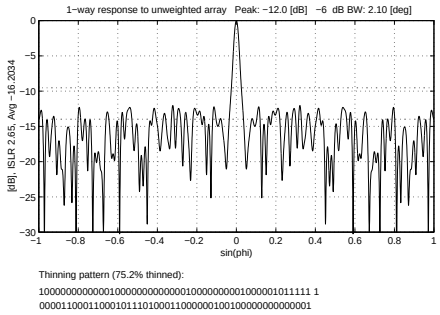

Figure 2: Array-pattern for optimized array with 25 elements out of 101, based on [3].

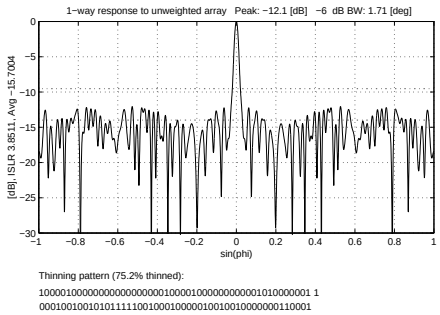

Figure 3: One-way array pattern for optimized array with 25 elements out of 101, optimized for minimum beamwidth.

and 4. The first represents a search over $2.0 \cdot 10^5$ configurations and has a sidelobe level of -12.06 dB and a -6 dB beamwidth of $1.71°$. The second solution is the result of a search over $2.9 \cdot 10^7$ configurations and it has a sidelobe level -12.36 dB and a -6 dB beamwidth of $2.10°$. Here the sidelobe level is improved by 0.32 dB over the reference. The other eight solutions are between the two given here, i.e. with some sidelobe level improvement and some beamwidth improvement over the reference.

Other solutions using the simulated annealing algorithm are shown in Figs. 5 and 6. If one accepts a widening of the mainlobe, the last figure shows that it is actually possible to find a sparse array that has a peak level which is equivalent to the mean value for a random sparse array ($1/K = -13.97$ dB). However in this case, most of the elements are clustered near the center of the array and the mainlobe suffers. The result of Fig. 5 is probably the best that can be obtained in terms of sidelobe level. The sidelobe level is in this case -13.2 dB which is 0.8 dB above the average level of a random array and corresponds to a level of $1.2/K$. This is our best estimate of the peak sidelobe level for algorithmically optimized 1D arrays. In [2] this level was estimated to be $1/K$, but that was for a small array where few of the elements was removed.

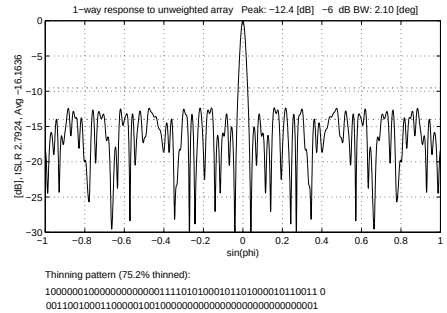

Figure 4: One-way array pattern for optimized array with 25 elements out of 101, optimized for minimum sidelobe level.

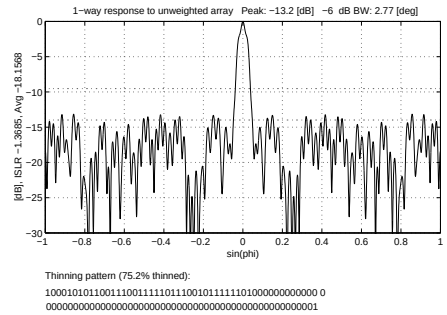

Figure 5: One-way array pattern for optimized array with 25 elements out of 101, peak sidelobe level -13.2 dB.

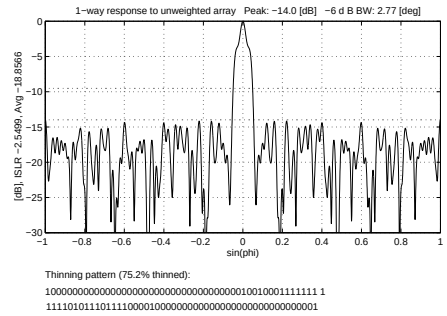

Figure 6: One-way array pattern for optimized array with 25 elements out of 101, peak sidelobe level -14 dB which is similar to the average array pattern for a random array.

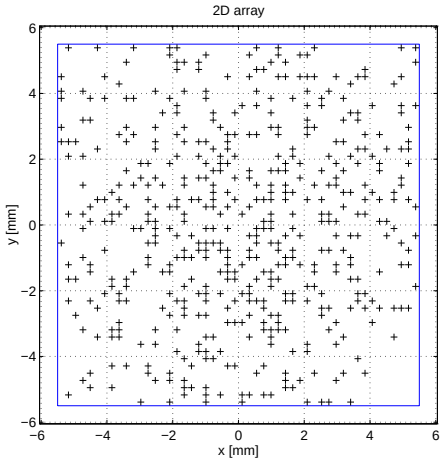

Figure 7: Element layout for optimized array with 500 elements out of 50 x 50, optimized for minimum beamwidth. The grid is shown for a frequency of 3.5 MHz and a velocity of sound of 1540 m/s corresponding to a wavelength of $\lambda = 0.44$ mm.

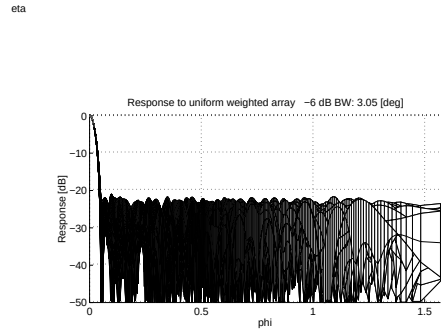

Figure 8: One-way array pattern for optimized array with 500 elements out of 50 x 50, optimized for minimum beamwidth. Peak sidelobe is -21.5 dB and the -6 dB beamwidth is $3.05°$. The array is steered to $\theta = \phi = 0$ and the response is shown as a function of azimuth angle and seen from the side in 3D space, i.e. the peak values over all elevation angles are seen.

5. 2D ARRAY OPTIMIZED WITH SIMULATED ANNEALING

The improved simulated annealing algorithm has been used for a 2D array of size 50 x 50 elements with element spacing $\lambda/2$. The algorithm with precomputed contributions from each element to the array pattern at all angles was used. For 64 points in the u and v directions, this requires several hundred Mbytes of RAM for storage. One of the best 500 element thinning patterns found is shown in Fig. 7. Finding this solutions took 46 hours on one CPU of a SGI Power Challenge using a MATLAB implementaiton of the simulated annealing algorithm.

The array pattern is shown in Fig. 8. It has a -6 dB beamwidth of $3.05°$ and a maximum sidelobe level of -21.5 dB. In comparison, the full array has a beamwidth of $2.81°$. The algorithm searched over 500 iterations and 5000 perturbations for each iteration, i.e. a total of 2.5 million out of a total of a $4 \cdot 10^{542}$ possible configurations.

6. CONCLUSION

In this paper we have presented an improved simulating annealing algorithm for optimization of the element layout of sparsely sampled arrays. The new implementation gives the algorithm more stochastic freedom and the increase in speed allows more configurations to be searched. For the classical layout problem of 25 elements on a 50λ aperture the improved algorithm has found a number of solutions that are better than the reference. When one accepts a widening of the mainlobe the algorithm has found solutions with peak sidelobe level of $1.2/K$ in the 1D case. The algorithm is also found successful in the 2D case giving element configurations with an almost totally flat sidelobe distribution.

7. REFERENCES

[1] M. I. Skolnik, G. Nemhauser, and J. W. Sherman III, "Dynamic programming applied to unequally spaced arrays," *IEEE Trans. Antennas Propagat.*, vol. AP-12, pp. 35–43, Jan. 1964.

[2] S. Holm, B. Elgetun, and G. Dahl, "Properties of the beampattern of weight- and layout-optimized sparse arrays," *IEEE Trans. Ultrason., Ferroelect., Freq. Contr.*, vol. 44, no. 5, pp. 983–991, Sept. 1997.

[3] V. Murino, A. Trucco, and C. S. Regazzoni, "Synthesis of unequally spaced arrays by simulated annealing," *IEEE Trans. Signal Processing*, vol. 44, pp. 119–123, Jan. 1996.

[4] A. Trucco and F. Repetto, "A stochastic approach to optimizing the aperture and the number of elements of an aperiodic array," in *Proc. OCEANS '96*, Sept. 1996, vol. 3, pp. 1510–1515.

[5] A. Trucco, "Thinning and weighting of large planar arrays by simulated annealing," *IEEE Trans. Ultrason., Ferroelect., Freq. Contr.*, vol. 46, no. 2, pp. 374–355, Mar. 1999.

[6] J.-F. Hopperstad, "Optimization of thinned arrays (in Norwegian)," M.S. thesis, Department of Informatics, University of Oslo, May 1998.

[7] Y. T. Lo and S. W. Lee, "A study of space-tapered arrays," *IEEE Trans. Antennas Propagat.*, vol. AP-14, pp. 22–30, Jan. 1966.

[8] R. K. Arora and N. C. V. Krishnamacharyulu, "Synthesis of unequally spaced arrays using dynamic programming," *IEEE Trans. Antennas Propagat.*, pp. 593–595, July 1968.

[9] S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, May 1983.

[10] John G. Proakis and Dimitris G. Manolakis, *Introduction to digital signal processing*, Macmillan, 1989, ISBN 0-02-396810-9.